

Component-based Software Engineering

Hands-on Exercise

Hands-on Exercise 1: Building a Component-Based E-Commerce API

- **Objective:**

- Develop a basic e-commerce API with components for managing products and orders
- Reinforcing CBSE principles such as modularity, reusability, and composition using Spring Boot.

Overview

We will build a simple e-commerce API that allows users to:

- Retrieve a list of products (GET /products).
- Add a new product (POST /products).

The application will consist of three main components:

- 1.Product Model:** Represents the data structure for products.
- 2.ProductRepository:** Handles data storage (in-memory for simplicity).
- 3.ProductService:** Encapsulates business logic.
- 4.ProductController:** Exposes REST endpoints.

Software Tools Needed

- **IDE:** IntelliJ IDEA, VS Code, or Eclipse with Spring Tools Suite.
- **Dependencies:** Spring Web, Spring Data JPA, H2 Database (via Spring Initializr).
- **Testing Tool:** Postman

Step 1: Set Up a Spring Boot Project

- Go to [Spring Initializr](#).
- Configure the project:
 - **Project:** Maven.
 - **Language:** Java.
 - **Spring Boot Version:** Latest stable (e.g., 3.4.4).
 - **Group:** com.example.
 - **Artifact:** ecommerce-api.
 - **Dependencies:** Add Spring Web, Spring Data JPA, and H2 Database.
- Generate and download the project, then open it in your IDE.
- Verify the project runs by executing the main class (EcommerceApiApplication.java) and checking for the Spring Boot banner in the console.

Step 2: Define a Product Model

Goal: Create a data model to represent products in the e-commerce system.

- Create a new package `com.example.ecommerceapi.model`.
- Add a `Product` class with fields for `id`, `name`, `price`, and `stock`.
- Include getters, setters, and a constructor.

Step 3: Create ProductRepository

Goal: Build a repository component to manage product data (in-memory for simplicity).

- Create a new package `com.example.ecommerceapi.repository`.
- Define a `ProductRepository` class annotated with `@Repository`.
- Implement basic CRUD operations: `findAll()` and `save()` using an in-memory List.

Step 4: Create ProductService

Goal: Implement a service component to handle business logic for product management.

- Create a new package `com.example.ecommerceapi.service`.
- Define a `ProductService` class annotated with `@Service`.
- Inject `ProductRepository` via constructor injection and implement methods to retrieve and add products.

Step 5: Create ProductController

Goal: Build a REST controller to expose product management endpoints.

- Create a new package `com.example.ecommerceapi.controller`.
- Define a `ProductController` class annotated with `@RestController`.
- Inject `ProductService` and implement GET and POST endpoints.

Step 6: Test the API Using Postman

- **Goal:** Verify the API works as expected.
- Start the Spring Boot application (run EcommerceApiApplication).
- Open Postman:
 - **GET Request:**
 - URL: `http://localhost:8080/products`
 - Method: GET
 - Expected Response: Empty list [] (initially).
 - **POST Request:**
 - URL: `http://localhost:8080/products`
 - Method: POST
 - Body (raw JSON):

```
json

{
  "name": "Laptop",
  "price": 999.99,
  "stock": 10
}
```
 - Expected Response: Product with an assigned id (e.g., `{"id": 1, "name": "Laptop", "price": 999.99, "stock": 10}`).
- Repeat the GET request to confirm the product was added.

Hands-on Exercise 1: Summary

- **Components:** ProductRepository, ProductService, and ProductController are modular, reusable units.
- **Interfaces:** Implicitly applied through Spring's dependency injection (e.g., ProductService could use an interface in future iterations).
- **Composition:** Dependency injection assembles the components into a cohesive API.
- **CBSE Principles:** Encapsulation (repository hides data storage), modularity (separate concerns), and reusability (components can be reused in other contexts).